

Lesson 16: Firebase Integration

What Firebase gives you

So far your app keeps data on one phone. But what if the user gets a new phone? What if two users need to share data? You need a backend -- a computer on the internet that stores data for your app.

Firebase is Google's backend made for app builders. It gives you two things beginners need most:

- Authentication -- sign users in with email and password, without writing your own login server.
- Firestore -- a database in the cloud where your app can save and read data from any device.

You do not need to manage servers. You click through a setup page, add a package, and write a few lines of code.

Setting it up (the short version)

The full setup has many clicks, so here is the map:

1. Go to the Firebase console (console.firebase.google.com) and create a project.
2. Add an Android app (and iOS if you need it) and download the config file into your Flutter project.
3. Add the packages to pubspec.yaml:

```
dependencies:  
  firebase_core: ^2.24.2  
  firebase_auth: ^4.15.3  
  cloud_firestore: ^4.13.6
```

4. In your main(), start Firebase before the app runs:

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp();  
  runApp(const MyApp());  
}
```

WidgetsFlutterBinding.ensureInitialized() is one line you just memorize: it lets you use await before runApp().

Signing a user in

With the setup done, signing in is two lines:

```
final credential = await FirebaseAuth.instance
```

```
.signInWithEmailAndPassword(email: email, password: password);
```

To create a new account instead, use `createUserWithEmailAndPassword` -- same shape, same two inputs.

To check who is signed in right now:

```
final user = FirebaseAuth.instance.currentUser;
if (user != null) {
  print('Signed in as ${user.email}');
}
```

And to sign out: `await FirebaseAuth.instance.signOut();`

Reading and writing data

Firestore stores data in collections (like folders) that hold documents (like files). A document is just a set of key-value pairs.

Writing a document:

```
await FirebaseFirestore.instance
  .collection('users')
  .doc('aarav')
  .set({'name': 'Aarav', 'score': 150});
```

Reading it back:

```
final doc = await FirebaseFirestore.instance
  .collection('users')
  .doc('aarav')
  .get();

print(doc['name']); // 'Aarav'
```

`collection('users')` picks the folder, `doc('aarav')` picks the file, `set()` writes, `get()` reads.

Because the data lives in the cloud, it is the same on every device the user signs in on.

One warning: by default anyone can read and write your database. Before you publish, set security rules in the Firebase console so users can only touch their own data.

Key terms

- Backend -- a computer on the internet that stores data and runs logic for your app.
- Firebase -- Google's backend service with ready-made login and database tools.
- Authentication -- proving who the user is (sign in / sign up).
- Firestore -- Firebase's cloud database of collections and documents.
- Collection -- a group of documents, like a folder.

- Document -- one record in Firestore, a set of key-value pairs.